

Financial RAG Pipeline

System Architecture and Project Workflow

Version: 1.0

Project Type: Retrieval-Augmented Generation (RAG) for Financial Documents

Purpose: Architecture Overview, Workflow Design, and Experiment Framework

Executive Summary

The Financial RAG Pipeline is a modular Retrieval-Augmented Generation (RAG) system designed to answer questions from financial documents such as SEC filings, annual reports, earnings reports, and other corporate disclosures.

The system evaluates how different retrieval strategies impact answer quality by comparing:

- Fixed-size chunking versus semantic chunking
- Similarity-based retrieval versus retrieval with cross-encoder reranking

To ensure objective evaluation, the platform executes four experimental configurations and measures both retrieval effectiveness and answer quality using standard information retrieval metrics and RAGAS evaluation metrics.

The architecture separates ingestion, retrieval, generation, and evaluation into independent modules, making the system scalable, maintainable, and easy to extend.

This project evaluated a Financial RAG pipeline using four financial document types: SEC filings, earnings call transcripts, insurance claims, and loan documents. Two chunking strategies (Fixed and Semantic) were compared, with and without a BGE reranker.

The best chunking strategy depends heavily on the specific nature of your documents and the level of detail required by your RAG application.

Choosing the right strategy is a balancing act: **small chunks** offer high precision for retrieval, while **large chunks** provide the rich context necessary for the LLM to generate meaningful answers.

The following strategies are recommended based on document type and complexity:

Fixed-Size Chunking: Useful for quick prototyping or simple documents where speed is a priority. It is recommended to use a **10–20% overlap** to ensure that facts split across boundaries aren't lost.

Semantic Chunking: Instead of relying on character counts, this technique uses embeddings to detect **topic shifts**. It is ideal for dense text where logical flow is more important than explicit paragraph markers.

Key Findings

Chunking Comparison

- **Fixed Chunking** produced 30 smaller chunks and achieved higher retrieval precision (86%) than Semantic Chunking (51%).
- **Semantic Chunking** generated only 12 larger chunks, which reduced retrieval precision but still contained sufficient information for accurate answers.
- Despite lower retrieval precision, Semantic Chunking delivered answer quality comparable to Fixed Chunking.

Reranking Impact

- Adding the **BGE reranker** consistently improved answer quality across both chunking methods.
- Reranking increased:
 - Mean Reciprocal Rank (MRR)
 - Faithfulness
 - Context Precision
- The highest Context Precision score (0.949) was achieved by **Semantic Chunking + Reranking**.

Performance Summary

Configuration	Strength
Fixed Chunking	Best retrieval precision
Fixed + Reranking	Best overall balance of accuracy and efficiency
Semantic Chunking	Better topic coherence for large documents
Semantic + Reranking	Highest context precision

Recommendation

For production deployment, **Fixed Chunking (500 characters with 50 overlap) combined with BGE Reranking** is recommended. This configuration delivered the highest faithfulness, strong retrieval performance, and the most predictable operational cost.

For long narrative documents such as SEC filings and earnings transcripts, **Semantic Chunking with Reranking** can be considered because it provides highly relevant context despite lower retrieval precision.

Limitations

The evaluation was conducted on a small dataset of only four documents. Because of the limited corpus size, Recall@5 reached 100% for all experiments and may not reflect real-world performance. Further validation on larger financial document collections is recommended before production implementation.

1. Project Objectives

The primary objectives of this project are:

Functional Objectives

- Ingest financial documents from multiple formats
- Generate embeddings and searchable vector indexes
- Retrieve relevant financial information efficiently
- Generate grounded answers using Large Language Models (LLMs)
- Provide source-backed responses

Research Objectives

- Evaluate the impact of chunking strategies
- Evaluate the impact of reranking techniques
- Identify the highest-performing retrieval architecture
- Measure answer quality and retrieval effectiveness

2. High-Level System Architecture

System Flow

Financial Documents

↓

Document Loading

↓

Metadata Normalization

↓

Chunk Generation

├— Fixed Chunking

└— Semantic Chunking

↓

Embedding Generation

↓

FAISS Vector Index Creation

├— Fixed Index

└— Semantic Index

↓

User Question

↓

Similarity Retrieval

↓

Optional Cross-Encoder Reranking

↓

Context Construction

↓

LLM Answer Generation

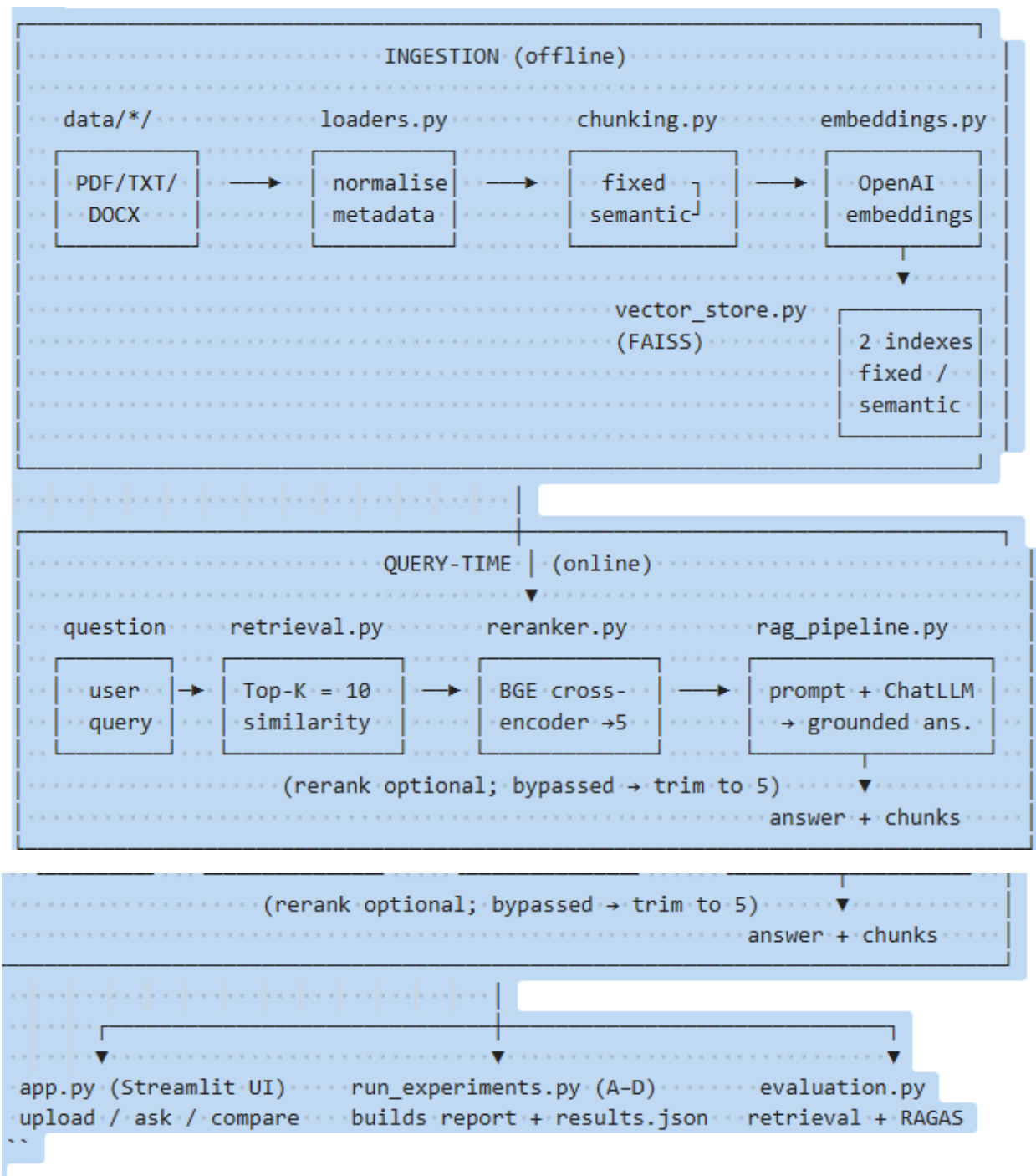
↓

Answer + Supporting Evidence

↓

Evaluation & Reporting

3. Architecture Layers



Layer 1: Data Ingestion

Purpose

Convert raw financial documents into standardized internal representations.

Inputs

- PDF Files
- TXT Files
- DOCX Files

Module

loaders.py

Responsibilities

- Read document contents
- Extract metadata
- Normalize file structures
- Create LangChain Document objects

Output

Standardized document objects containing:

- Content
- Source
- Page number
- Document type

Layer 2: Chunk Processing

Purpose

Transform large documents into retrievable chunks.

Module

chunking.py

Strategy A: Fixed Chunking

Uses:

RecursiveCharacterTextSplitter

Characteristics:

- Fixed chunk size
- Fixed overlap
- Simple implementation
- Baseline comparison

Advantages:

- Fast processing
- Predictable chunk sizes

Limitations:

- May split important context
 - Ignores semantic boundaries
-

Strategy B: Semantic Chunking

Uses:

SemanticChunker

Characteristics:

- Context-aware splitting
- Preserves topic boundaries
- Produces meaningful retrieval units

Advantages:

- Better context preservation
- Improved retrieval relevance

Limitations:

- Higher processing cost
 - More complex implementation
-

Layer 3: Embedding Generation

Purpose

Convert text chunks into vector representations.

Module

embeddings.py

Default Model

OpenAI text-embedding-3-large

Process

Text Chunk

↓

Embedding Model

↓

Dense Vector Representation

Output

Embeddings suitable for similarity search.

Layer 4: Vector Database Layer

Purpose

Store embeddings and enable efficient retrieval.

Module

vector_store.py

Technology

FAISS

Index Design

Two independent indexes are maintained:

Fixed Chunk Index

indexes/faiss_fixed

Semantic Chunk Index

indexes/faiss_semantic

Design Benefits

- Fair experimental comparison
 - Independent retrieval testing
 - Reduced configuration coupling
-

Layer 5: Retrieval Layer

Purpose

Find the most relevant document chunks for a query.

Module

retrieval.py

Workflow

User Question

↓

Question Embedding

↓

FAISS Similarity Search

↓

Top 10 Chunks

Output

RetrievedChunk objects containing:

- Chunk text
 - Similarity score
 - Metadata
-

Layer 6: Reranking Layer

Purpose

Improve retrieval precision before generation.

Module

reranker.py

Technology

BAAI/bge-reranker-large

Workflow

Top 10 Retrieved Chunks

↓

Cross-Encoder Scoring

↓

Rank by Relevance

↓

Top 5 Chunks

Expected Benefits

- Improved relevance
- Better context selection
- Reduced retrieval noise

Layer 7: Generation Layer

Purpose

Generate grounded answers from retrieved context.

Module

rag_pipeline.py

Workflow

Retrieved Context

↓

Prompt Construction

↓

ChatOpenAI

↓

Grounded Answer

Prompt Rules

The model must:

- Use only provided context
- Avoid hallucinations
- Return:

"Information not found in provided documents."

when evidence is unavailable.

Output

RAGResult

Containing:

- Question
- Generated Answer
- Retrieved Context
- Metadata

Layer 8: User Interface Layer

Technology

Streamlit

Module

app.py

Features

- Document upload
- Index creation
- Query interface

- Configuration controls
 - Strategy comparison view
-

4. Query Execution Workflow

Runtime Process

Step 1:

User submits a question.

↓

Step 2:

System embeds the query.

↓

Step 3:

FAISS retrieves Top 10 similar chunks.

↓

Step 4:

Optional reranking selects the best 5 chunks.

↓

Step 5:

Context is formatted for the LLM.

↓

Step 6:

Prompt is submitted to GPT-4o-mini.

↓

Step 7:

Generated answer and supporting evidence are returned.

5. Experimental Framework

The architecture is designed around four controlled experiments.

Experiment Matrix

Experiment Chunking Strategy Reranking

A	Fixed	Disabled
B	Fixed	Enabled
C	Semantic	Disabled
D	Semantic	Enabled

Experimental Goals

Impact of Reranking

A → B

Measures reranking effectiveness on fixed chunks.

C → D

Measures reranking effectiveness on semantic chunks.

Impact of Chunking

A → C

Measures retrieval improvements from semantic chunking.

B → D

Measures combined improvements with reranking enabled.

6. Evaluation Framework

The project evaluates both retrieval quality and answer quality.

Retrieval Evaluation

Module

evaluation.py

Metrics

Precision@5

Measures the proportion of retrieved chunks that are relevant.

Recall@5

Measures how many relevant documents were retrieved.

Mean Reciprocal Rank (MRR)

Measures how quickly relevant documents appear in ranked results.

Benefits

- Deterministic
 - Reproducible
 - Low computational cost
-

Answer Quality Evaluation

Framework

RAGAS

Metrics

Faithfulness

Measures whether answers are supported by retrieved context.

Answer Relevancy

Measures whether answers address the user's question.

Context Precision

Measures relevance of supplied context.

Context Recall

Measures completeness of retrieved context.

Benefits

- End-to-end evaluation
 - Measures actual user experience
 - Captures retrieval-generation interactions
-

7. Key Architectural Decisions

Separate FAISS Indexes

Rationale:

- Prevents cross-contamination
 - Enables direct comparisons
 - Simplifies experimentation
-

Centralized Configuration

All parameters reside in config.py.

Benefits:

- Reproducibility
 - Simplified tuning
 - Easier deployment
-

Lazy Loading

Heavy dependencies load only when required.

Examples:

- SemanticChunker
- Cross Encoder
- RAGAS

Benefits:

- Faster startup

- Lower memory usage
-

Similarity Normalization

FAISS distances are converted into similarity scores:

Similarity = $1 / (1 + \text{Distance})$

Benefits:

- Easier interpretation
 - Consistent UI presentation
-

Shared Data Contracts

Common data structures:

- RetrievedChunk
- RAGResult

Benefits:

- Loose coupling
 - Easier maintenance
 - Cleaner integrations
-

Resource Caching

Cached components include:

- Embedding models
- Reranking models
- Streamlit resources

Benefits:

- Reduced latency
- Improved scalability

- Better user experience

8. Technology Stack

Category	Technology
Framework	LangChain
Embeddings	OpenAI text-embedding-3-large
Vector Database	FAISS
Reranker	BAAI/bge-reranker-large
LLM	GPT-4o-mini
Evaluation	RAGAS
UI	Streamlit

9. Expected Deliverables

Technical Deliverables

- Financial document ingestion pipeline
- Fixed chunk retrieval index
- Semantic chunk retrieval index
- RAG question-answering engine
- Cross-encoder reranking module
- Evaluation framework

Reports

- results.json
 - comparison_report.md
-

User Interface

Streamlit dashboard supporting:

- Document upload
 - Index creation
 - Query execution
 - Configuration management
 - Experiment comparison
-

10. Conclusion

The Financial RAG Pipeline serves as both a production-ready question-answering system and an experimental benchmarking platform.

By systematically comparing fixed versus semantic chunking and retrieval-only versus reranked retrieval, the project provides quantitative insights into how retrieval architecture influences answer quality for financial document analysis.

The modular design allows future enhancements such as hybrid search, alternative embedding models, vector databases, and advanced reranking strategies while maintaining a consistent evaluation framework.